

How We Deleted \$600K in Software Costs

And What We Learned Along the Way

by Ryan Clements, Owner of [Byte Bot](#)

A business with a deadline doesn't care about your architecture...

A business with a deadline doesn't care about your architecture... It only cares that you deliver lasting results.

Goals

Goals

📖 Share a **war story** about replacing a \$600K/year vendor

Goals

- 📖 Share a **war story** about replacing a \$600K/year vendor
- 🎯 Learn how to **hyperfocus** on driving tangible results

Goals

-  Share a **war story** about replacing a \$600K/year vendor
-  Learn how to **hyperfocus** on driving tangible results
-  Learn **effective** technical design and project management techniques

Goals

-  Share a **war story** about replacing a \$600K/year vendor
-  Learn how to **hyperfocus** on driving tangible results
-  Learn **effective** technical design and project management techniques
-  Learn how to **manage** disincentivized vendors

Goals

-  Share a **war story** about replacing a \$600K/year vendor
-  Learn how to **hyperfocus** on driving tangible results
-  Learn **effective** technical design and project management techniques
-  Learn how to **manage** disincentivized vendors
-  Learn practical **design principles** for ETL pipelines

Goals

- 📖 Share a **war story** about replacing a \$600K/year vendor
- 🎯 Learn how to **hyperfocus** on driving tangible results
- 📝 Learn **effective** technical design and project management techniques
- 👊 Learn how to **manage** disincentivized vendors
- 🔧 Learn practical **design principles** for ETL pipelines
- 🧠 Learn how to **leverage** constraints for better outcomes

Who am I?

RC Clements, Owner of **Byte Bot**

 bytebot.io

 info@bytebot.io

 [@RyanClementsHax](https://twitter.com/RyanClementsHax)

 bytebot.beehiiv.com/subscribe



Who am I?

RC Clements, Owner of **Byte Bot**

a full-stack agency that establishes best practices in teams through staff augmentation and coaching

 bytebot.io

 info@bytebot.io

 [@RyanClementsHax](https://twitter.com/RyanClementsHax)

 bytebot.beehiiv.com/subscribe



Who am I?

RC Clements, Owner of **Byte Bot**

a full-stack agency that establishes best practices in teams through staff augmentation and coaching



 bytebot.io

 info@bytebot.io

 [@RyanClementsHax](https://twitter.com/RyanClementsHax)

 bytebot.beehiiv.com/subscribe





The Client's Origin Story

The Client's Origin Story



Scrappy start

The Client's Origin Story



Scrappy start



Launched fast with off the shelf tools

The Client's Origin Story



Scrappy start

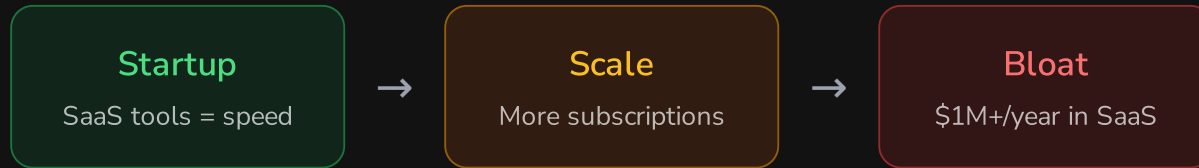


Launched fast with off the shelf tools



Getting rated as a loan servicer was existential

Startup → Scale → Bloat



The \$1M Problem



The \$1M Problem

📦 \$1M+/year in subscriptions to just one vendor

The \$1M Problem

🔗 \$1M+/year in subscriptions to just one vendor

📊 Using maybe 20% of features

The \$1M Problem

🔗 \$1M+/year in subscriptions to just one vendor

📊 Using maybe 20% of features

🙄 Sorta worked

The \$1M Problem

📁 \$1M+/year in subscriptions to just one vendor

📊 Using maybe 20% of features

🙄 Sorta worked

🔒 Locked into contracts

The \$1M Problem

✂️ \$1M+/year in subscriptions to just one vendor

📊 Using maybe 20% of features

🙄 Sorta worked

🔒 Locked into contracts

🤔 Major opportunity cost

The Specific Pain Point

The Specific Pain Point

Servicer

The Specific Pain Point

Servicer

DCH Parent Co

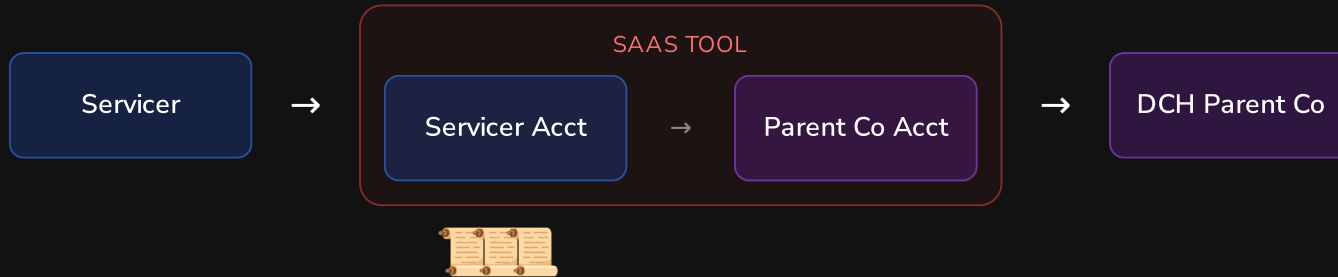
The Specific Pain Point



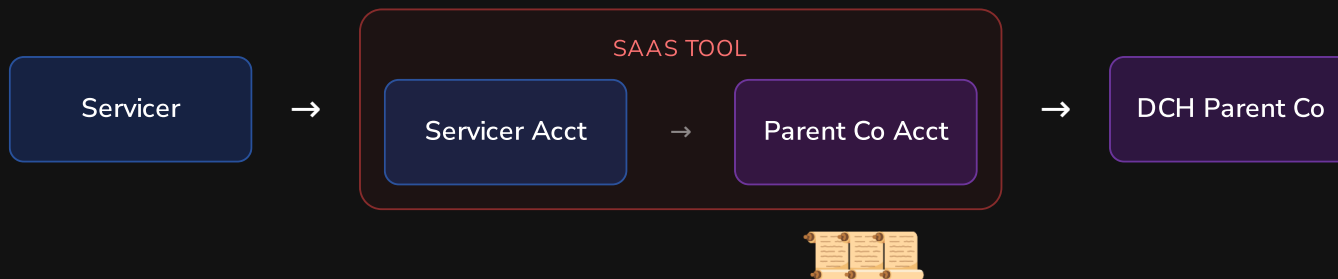
The Specific Pain Point



The Specific Pain Point



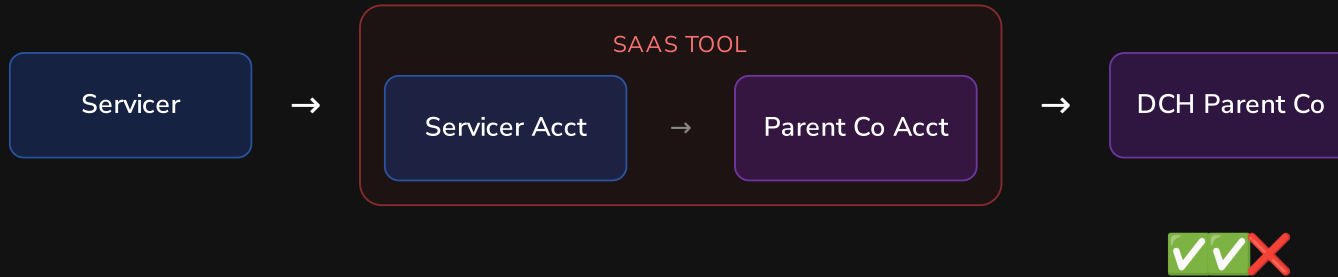
The Specific Pain Point



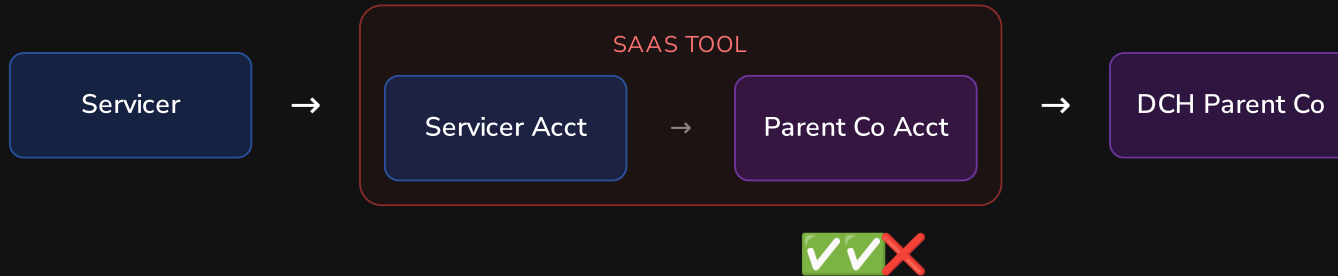
The Specific Pain Point



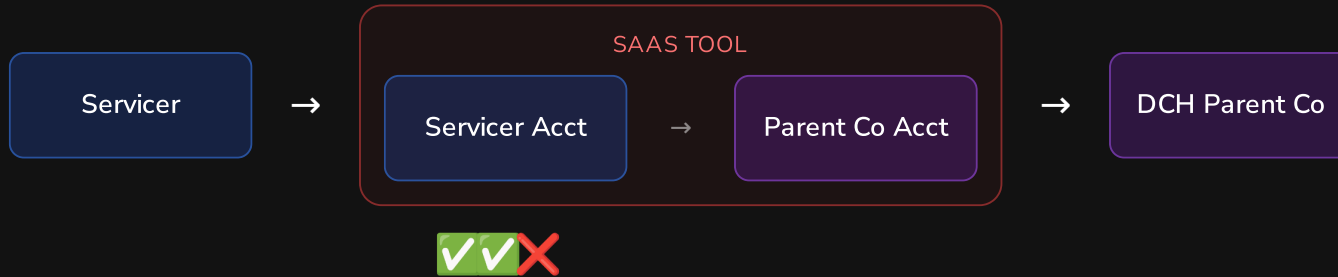
The Specific Pain Point



The Specific Pain Point



The Specific Pain Point



The Specific Pain Point



The Insight

The Insight

This workflow didn't **need** to live in an external tool.

The Insight

This workflow didn't **need** to live in an external tool.

The parent company **developed an internal platform** since the servicer was founded.

The Insight

This workflow didn't **need** to live in an external tool.

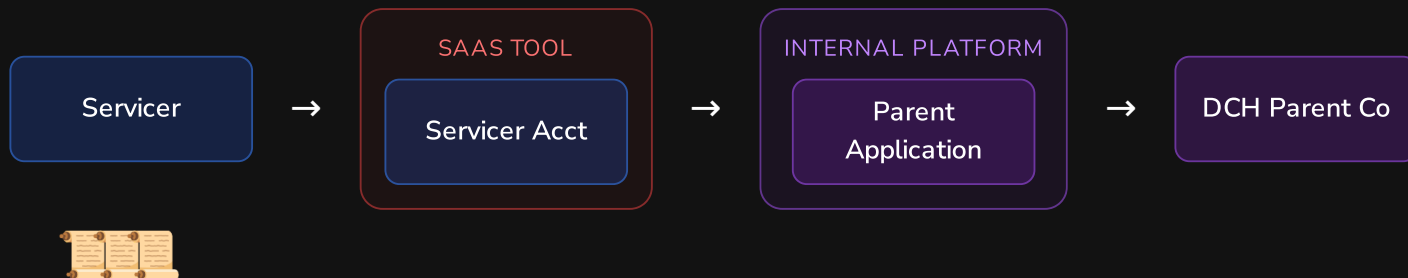
The parent company **developed an internal platform** since the servicer was founded.

It could be replicated **in-house**.

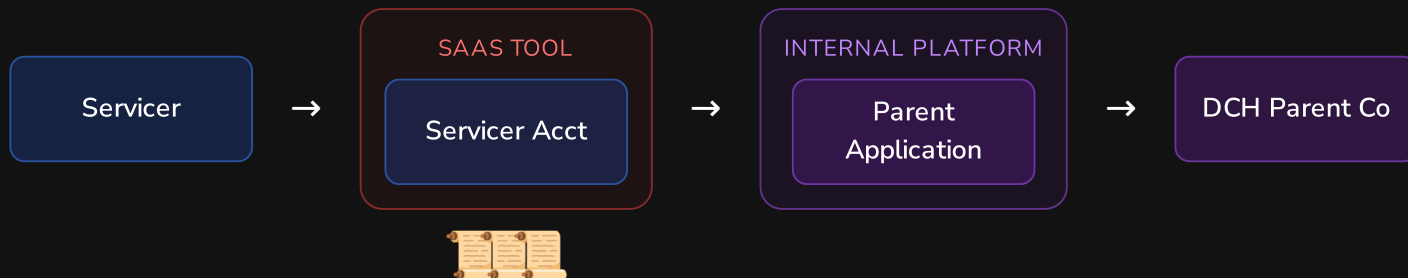
The Vision



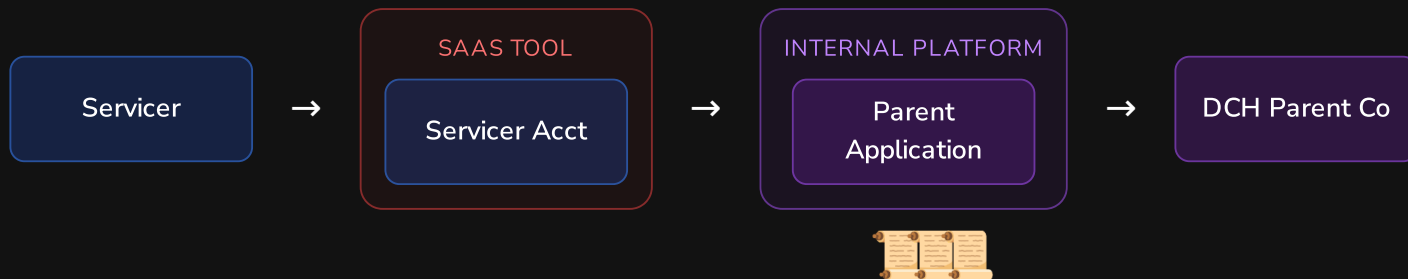
The Vision



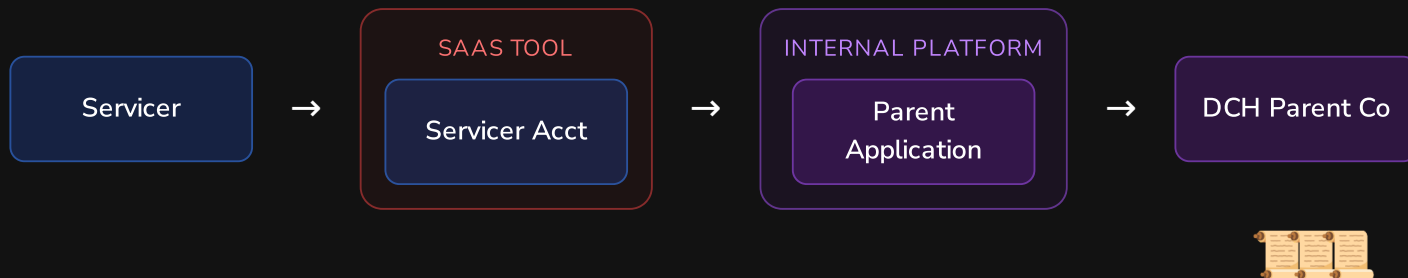
The Vision



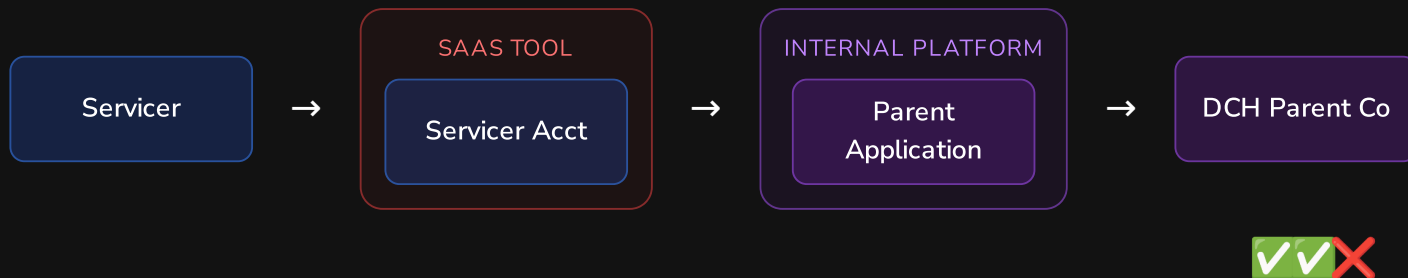
The Vision



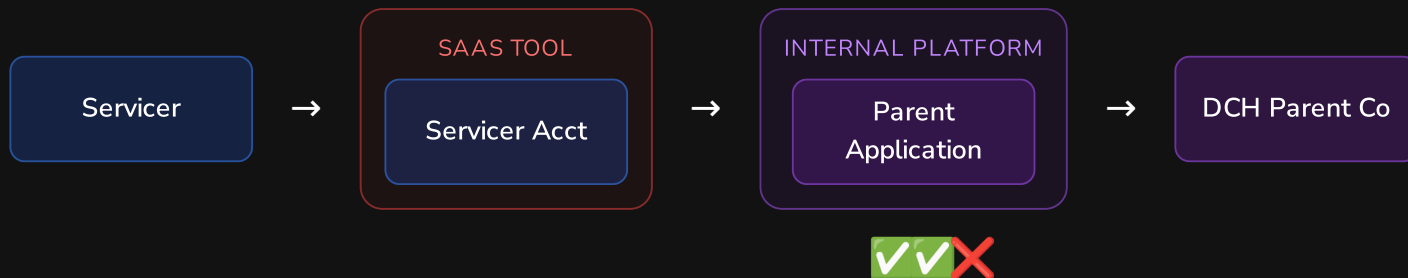
The Vision



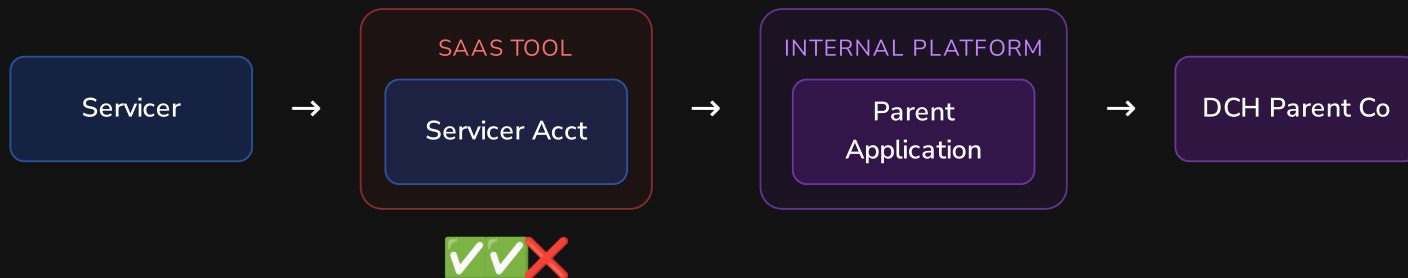
The Vision



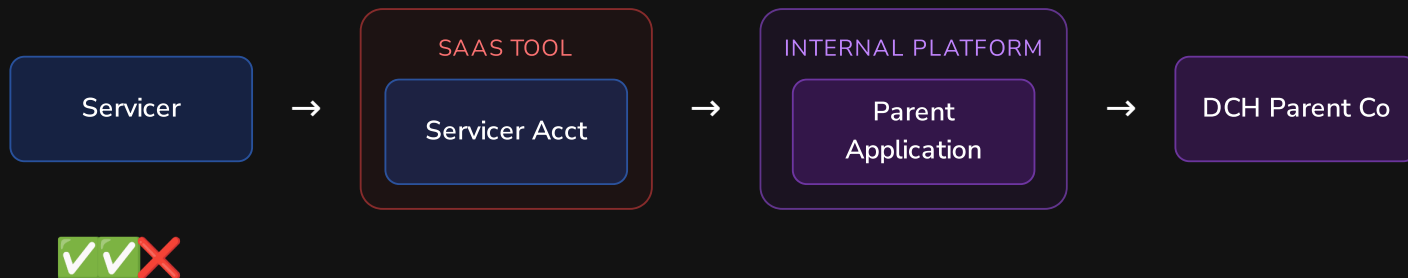
The Vision



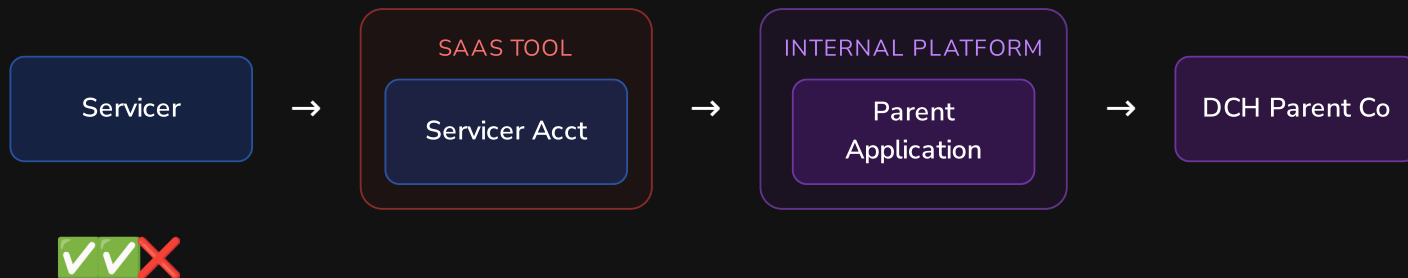
The Vision



The Vision



The Vision



Same workflow, **replicated in-house** on the internal platform.

The Deadline

The Deadline

4 months

The Deadline

4 months

Starting now

The Deadline

4 months

Starting now 🤔

Let's Begin

Step 1: Define Before You Build

The ABDiff Framework

The ABDiff Framework

A

Current State

The ABDiff Framework

A

Current State

B

Target State

The ABDiff Framework



The ABDiff Framework



This is what you ship

The ABDiff Framework

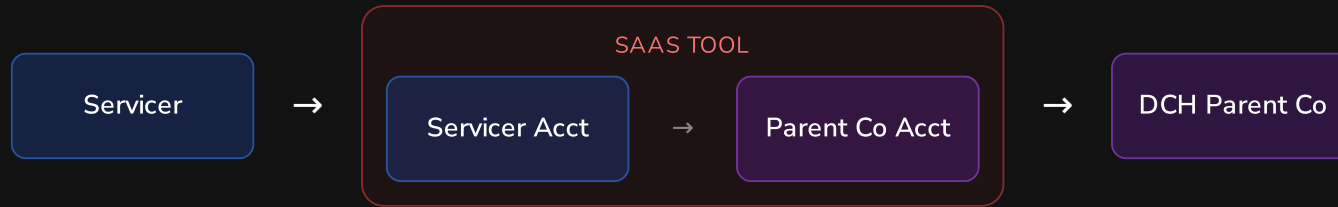


This is what you ship

bytebot.io/blog/stop-assuming-alignment-start-proving-it

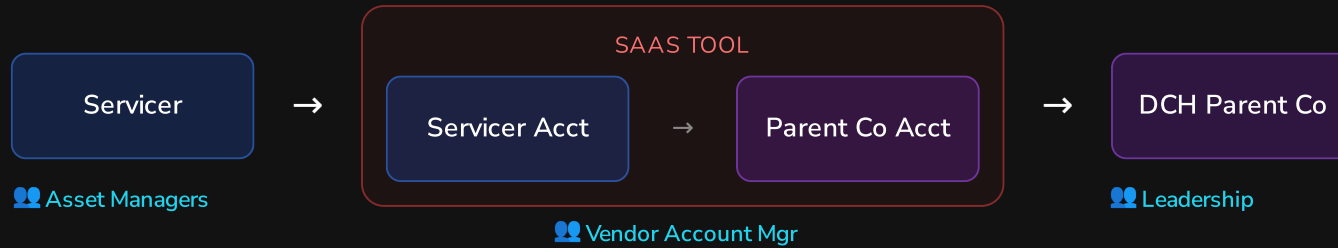
Point A - Current State

Point A - Current State



📄 Documents flow right → ✓✗ Decisions flow left

Point A - Current State



Point A - Current State



Point A - Current State



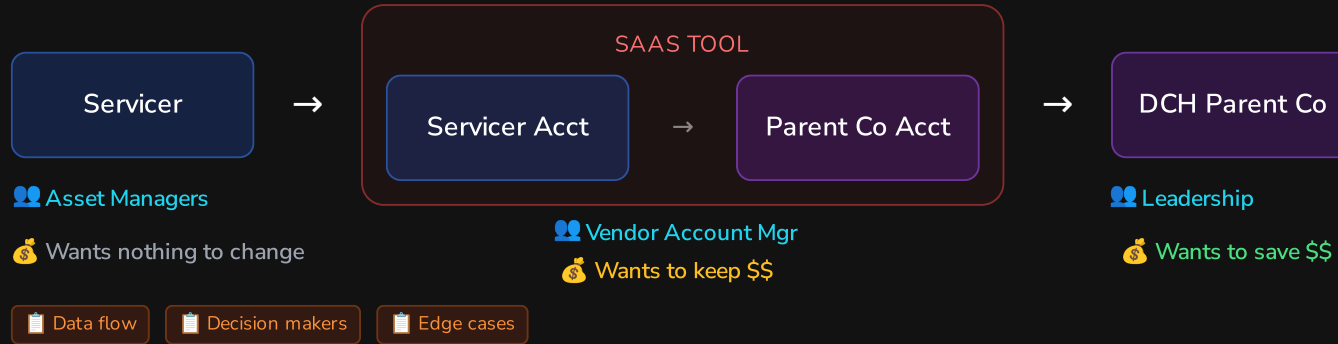
Hard deadline: 4 months - no extensions



Point A - Current State



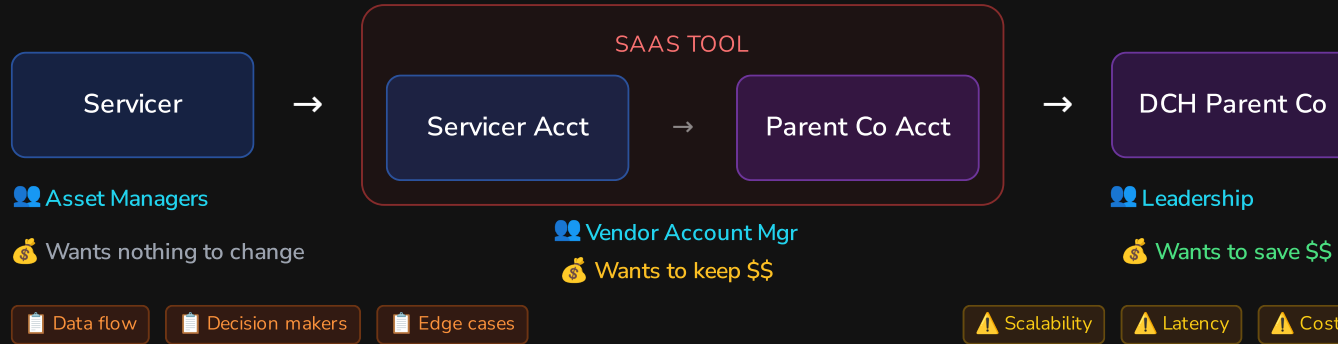
Hard deadline: 4 months - no extensions



Point A - Current State



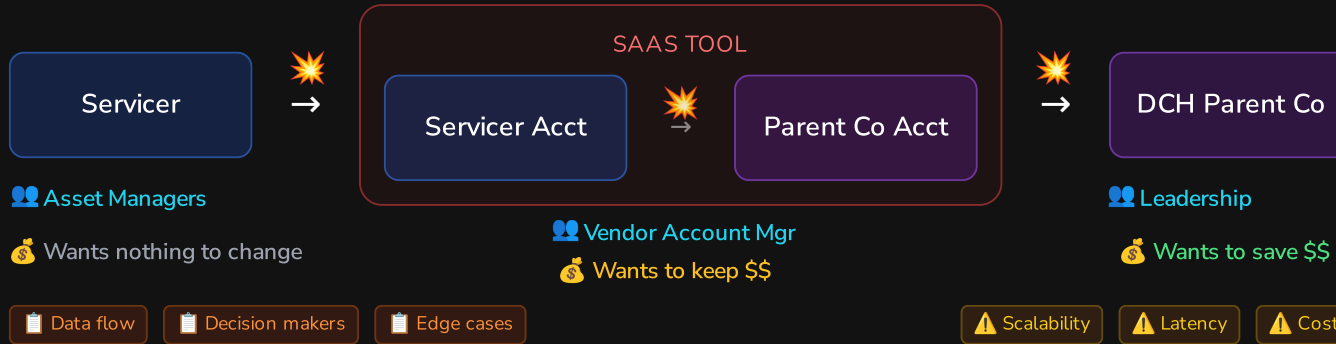
Hard deadline: 4 months - no extensions



Point A - Current State



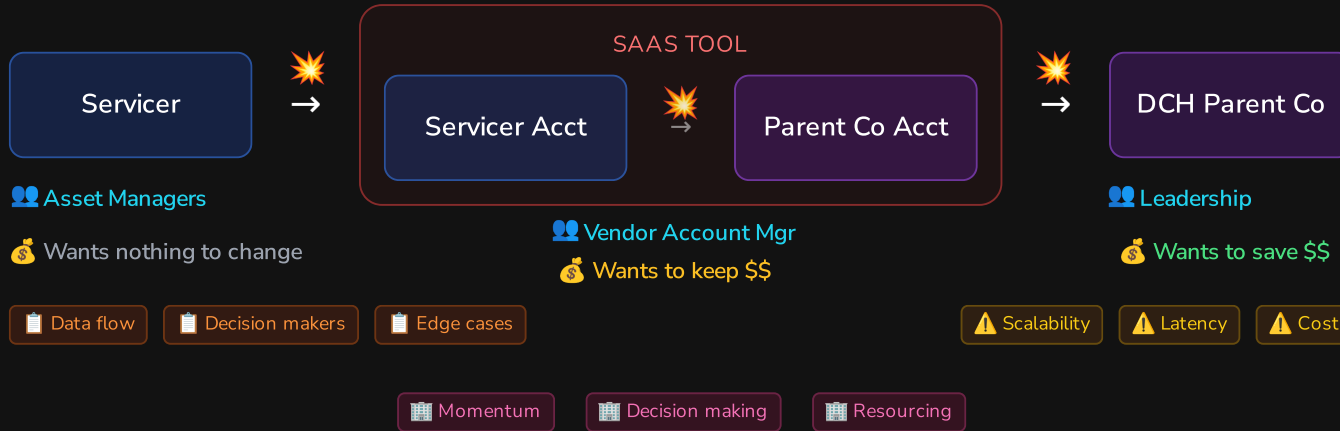
Hard deadline: 4 months - no extensions



Point A - Current State



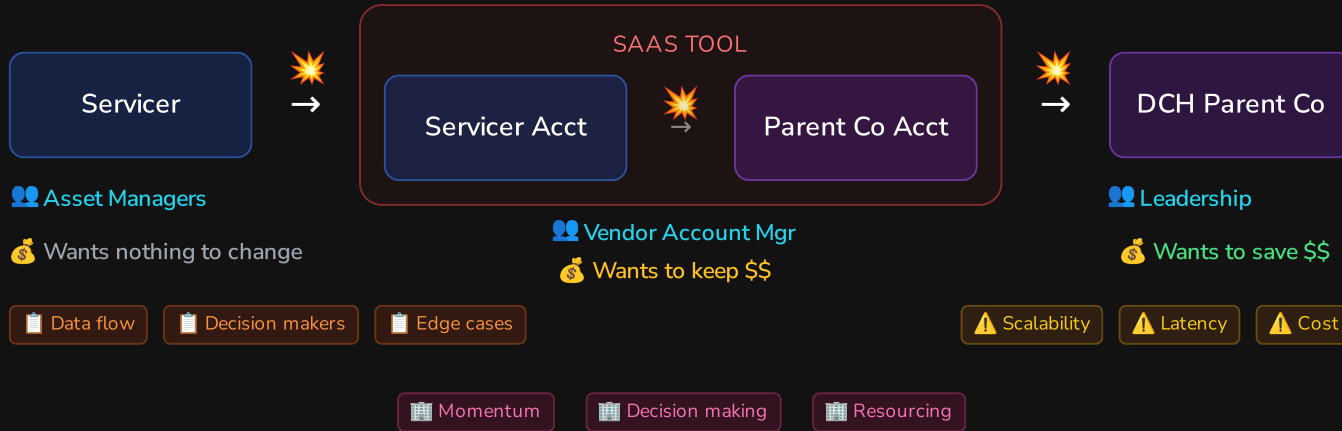
Hard deadline: 4 months - no extensions



Point A - Current State



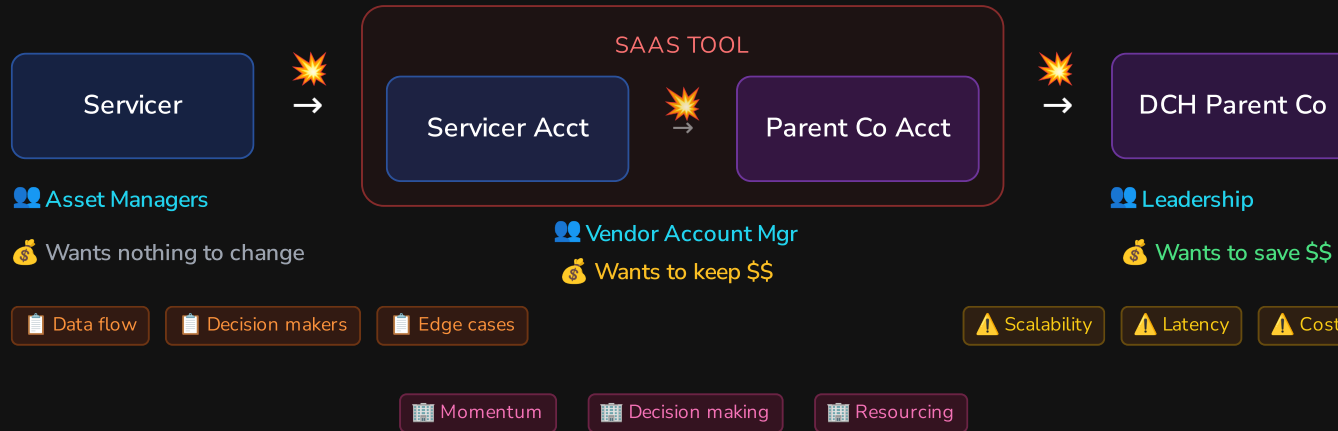
Hard deadline: 4 months - no extensions



Understanding the tech $\neq$ understanding Point A

Point A - Current State

 Hard deadline: 4 months - no extensions



If no one's listening to your ideas, this is probably why.

There are known knowns...

There are known knowns...

...and then there are the APIs you've never integrated with.

Major Risks

Major Risks

1. Poorly documented APIs

Major Risks

1. Poorly documented APIs
2. A vendor with zero incentive to help

Major Risks

1. Poorly documented APIs
2. A vendor with zero incentive to help
3. Data issues

The Data Problem

The Data Problem



Messy data flows

The Data Problem



Messy data flows



No clean IDs

The Data Problem



Messy data flows



No clean IDs



"Creative" data quality

The Data Problem



Messy data flows



No clean IDs



"Creative" data quality

"Don't trust. Verify. Then verify again."

The Vendor Problem

How do we get them to do stuff for us?

The Vendor Problem

How do we get them to do stuff for us? 🙋

Point B - Target State



📄 Documents flow right → ✓✗ Decisions flow left

Point B - Target State



Point B is NOT...

Point B is NOT...

 A microservice

Point B is NOT...

 A microservice

 An architecture

Point B is NOT...

 A microservice

 An architecture

 A policy or process

Point B is NOT...

 A microservice

 An architecture

 A policy or process

 A list of deliverables

Point B is NOT...



A microservice



An architecture



A policy or process



A list of deliverables

These are **outputs**, not **outcomes**.

Point B IS...

Point B IS...



The why behind the work

Point B IS...

 The why behind the work

 The impact on the business

Point B IS...



The why behind the work



The impact on the business



The outcome that matters

Point B IS...

 The **why** behind the work

 The **impact** on the business

 The **outcome** that matters

If no one gave you the **why**, go find it.

Point B



Point B



Point B

Save \$50k/mo.

Point B

Save \$50k/mo. Minimal disruption.

Point B

Save \$50k/mo. Minimal disruption. Don't be late.

Point B

Save \$50k/mo. Minimal disruption. Don't be late.

That's it. That's point B.

The Diff = The Backlog

The Diff = The Backlog

 Strategy — which approach and why

The Diff = The Backlog



Strategy — which approach and why



Architecture — systems, boundaries, contracts

The Diff = The Backlog



Strategy — which approach and why



Architecture — systems, boundaries, contracts



Subtasks — discrete units with clear owners

The Diff = The Backlog



Strategy — which approach and why



Architecture — systems, boundaries, contracts



Subtasks — discrete units with clear owners



Integrations — every touchpoint between systems

The Diff = The Backlog



Strategy — which approach and why



Architecture — systems, boundaries, contracts



Subtasks — discrete units with clear owners



Integrations — every touchpoint between systems



Migrations — data, schemas, feature flags

The Diff = The Backlog

-  Strategy — which approach and why
-  Architecture — systems, boundaries, contracts
-  Subtasks — discrete units with clear owners
-  Integrations — every touchpoint between systems
-  Migrations — data, schemas, feature flags
-  Testing — what to validate and how

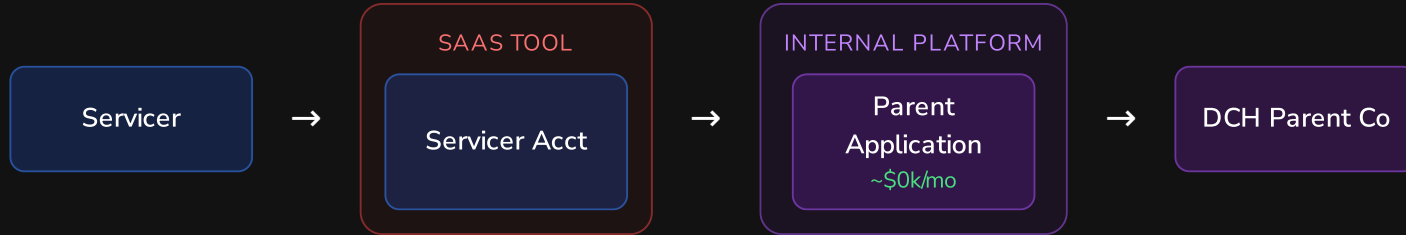
The Diff = The Backlog

-  Strategy — which approach and why
-  Architecture — systems, boundaries, contracts
-  Subtasks — discrete units with clear owners
-  Integrations — every touchpoint between systems
-  Migrations — data, schemas, feature flags
-  Testing — what to validate and how
-  Rollout — deployment steps, rollback plans

The Diff for Our Project

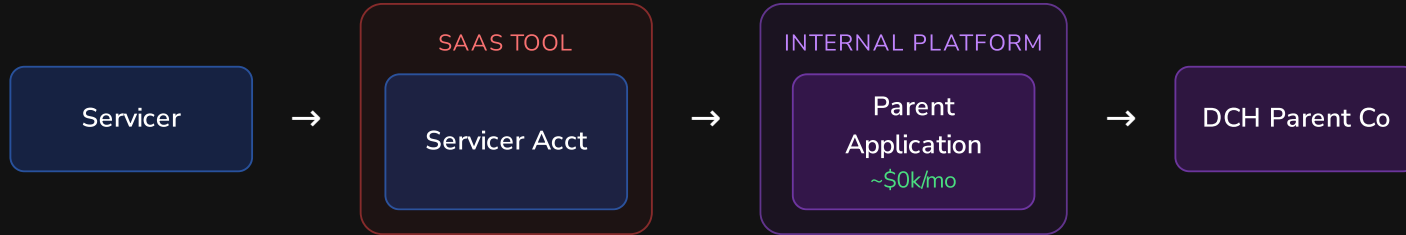


The Diff for Our Project



Hot swap the parent account

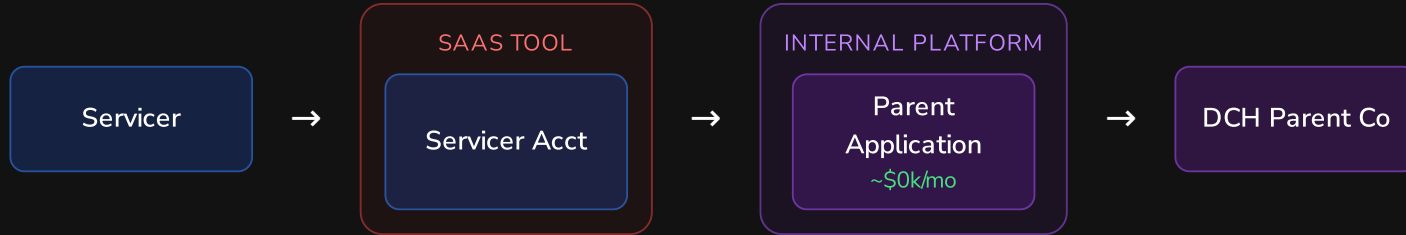
The Diff for Our Project



↺ Hot swap the parent account

🐛 Bug-for-bug compatible

The Diff for Our Project

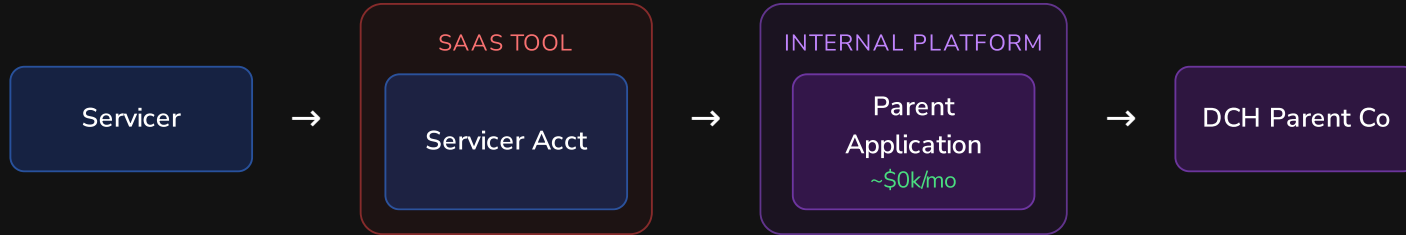


🔄 Hot swap the parent account

🐛 Bug-for-bug compatible

📦 Backfill old data

The Diff for Our Project



Hot swap the parent account



Bug-for-bug compatible



Backfill old data



Monitor and alert

Key Decisions

Key Decisions



Simple tools

Key Decisions



Simple tools



Cancel the contract - nothing else

Key Decisions



Simple tools



Cancel the contract - nothing else



Sources of truth

Key Decisions



Simple tools



Cancel the contract - nothing else



Sources of truth



Idempotency

Key Decisions



Simple tools



Cancel the contract - nothing else



Sources of truth



Idempotency



Logging over metrics

Key Decisions



Simple tools



Cancel the contract - nothing else



Sources of truth



Idempotency



Logging over metrics

"Don't try to solve every problem. We stayed focused."

Step 2: Get started...almost

Managing the Disincentivized Vendor

The Communication Breakthrough

TODO: visual showing emails going into a void vs. a Teams channel lighting up - the "before and after" of communication

The Communication Breakthrough

TODO: visual showing emails going into a void vs. a Teams channel lighting up - the "before and after" of communication



The Communication Breakthrough

TODO: visual showing emails going into a void vs. a Teams channel lighting up - the "before and after" of communication



The "Ball in Their Court" Strategy

"They had zero incentive to meet our deadline. So we made sure they were never waiting on us."

The "Ball in Their Court" Strategy

"They had zero incentive to meet our deadline. So we made sure they were never waiting on us."

⚡ Respond fast

The "Ball in Their Court" Strategy

"They had zero incentive to meet our deadline. So we made sure they were never waiting on us."

⚡ Respond fast

🔮 Pre-answer questions

The "Ball in Their Court" Strategy

"They had zero incentive to meet our deadline. So we made sure they were never waiting on us."

⚡ Respond fast

🌐 Pre-answer questions

📝 Document everything

The "Ball in Their Court" Strategy

"They had zero incentive to meet our deadline. So we made sure they were never waiting on us."

⚡ Respond fast

🔮 Pre-answer questions

📝 Document everything

🔦 Make delays visible

Security - Early

Security - Early

 Day one

Security - Early

 Day one

 Access, SFTP, credentials

Security - Early

 Day one

 Access, SFTP, credentials

 Hit the ground running

"Their API took 15 minutes to respond."

"Their API took 15 minutes to respond."

I wish I was joking.

Local DX Investment

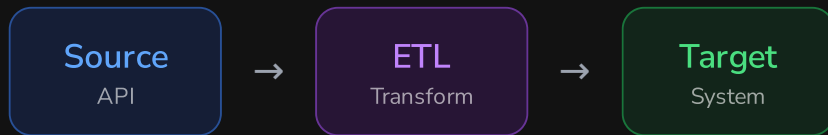


Local DX Investment

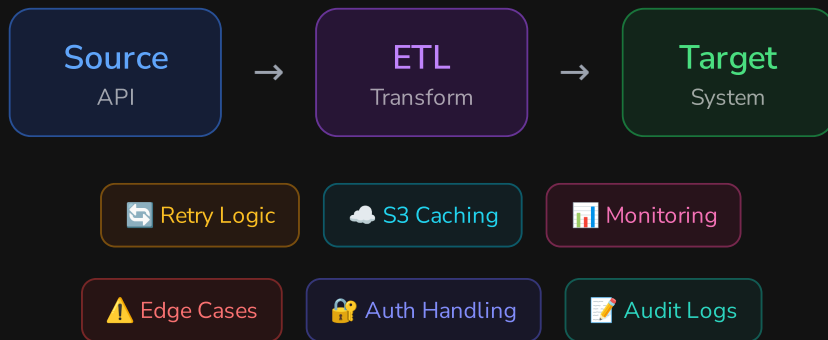


✨ Still used today - not just for dev, but for production debugging

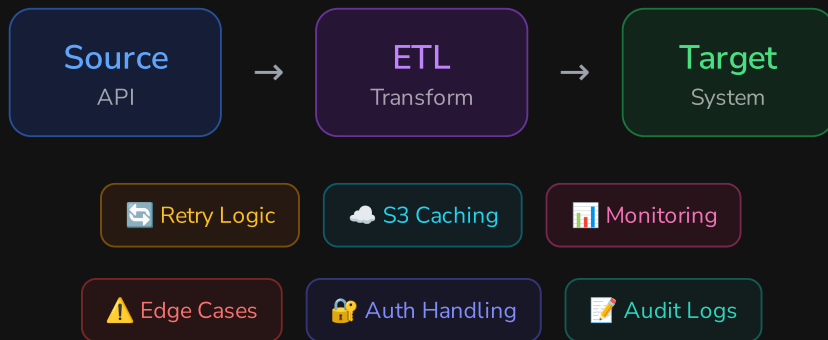
"Get the bones working before you add the muscle."



"Get the bones working before you add the muscle."



"Get the bones working before you add the muscle."



🦴 Ship the skeleton → 💪 Add the muscle

Step 3: Implementing a Good Design

What is an ETL?

Source

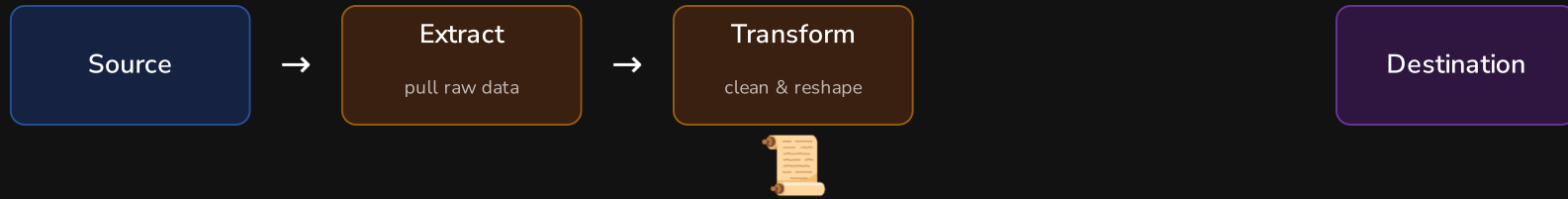


Destination

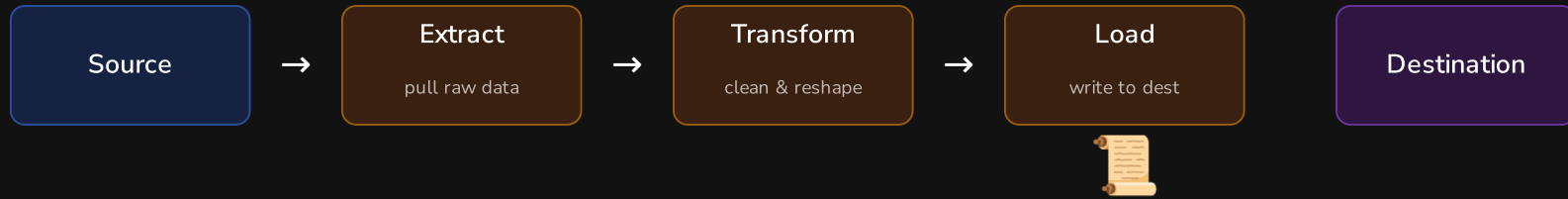
What is an ETL?



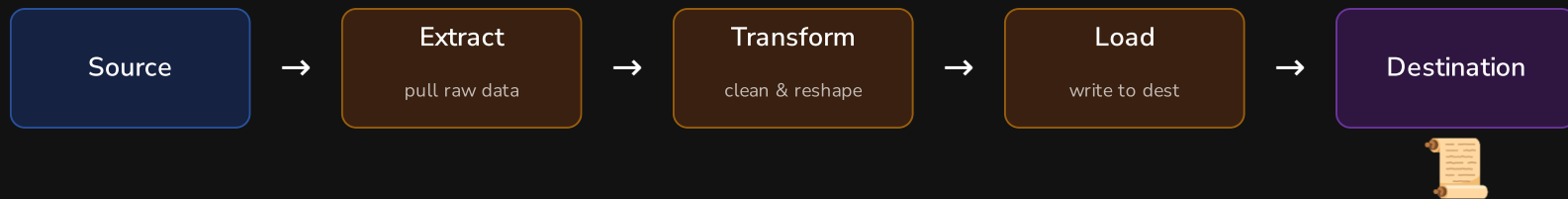
What is an ETL?



What is an ETL?



What is an ETL?



Extract the data. Transform it. Load it somewhere useful.

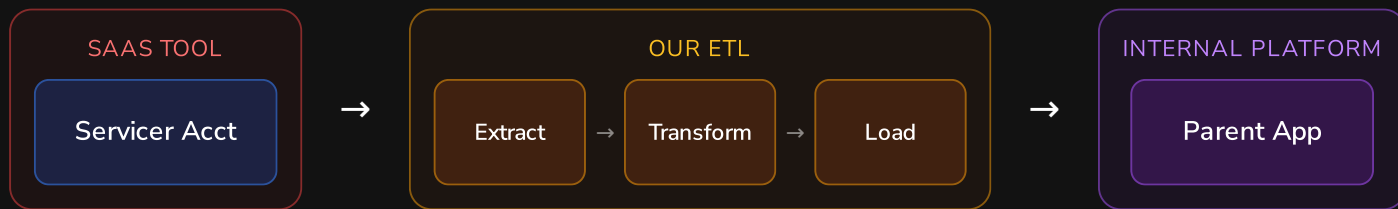
Our ETL



Our ETL



Our ETL

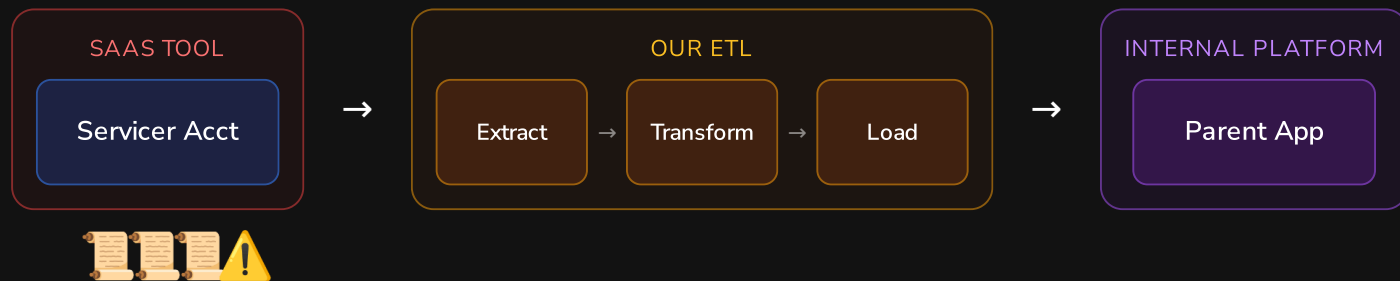


A small, focused pipeline bridging two systems.

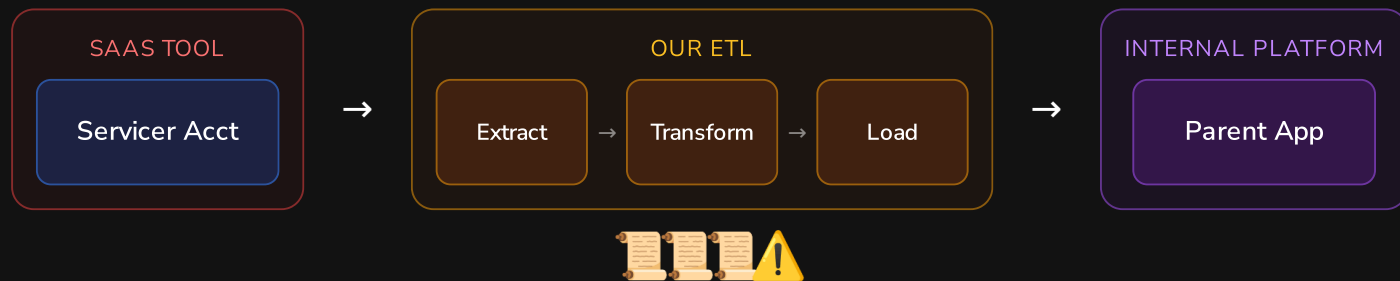
Data Quality Skepticism



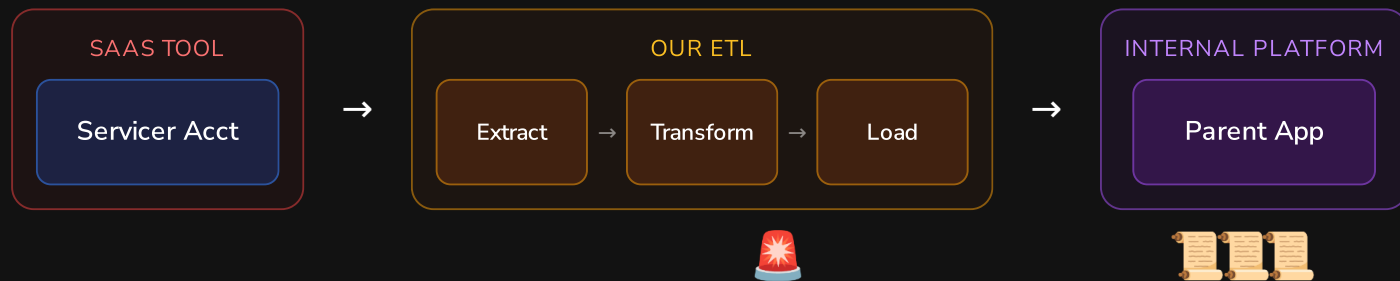
Data Quality Skepticism



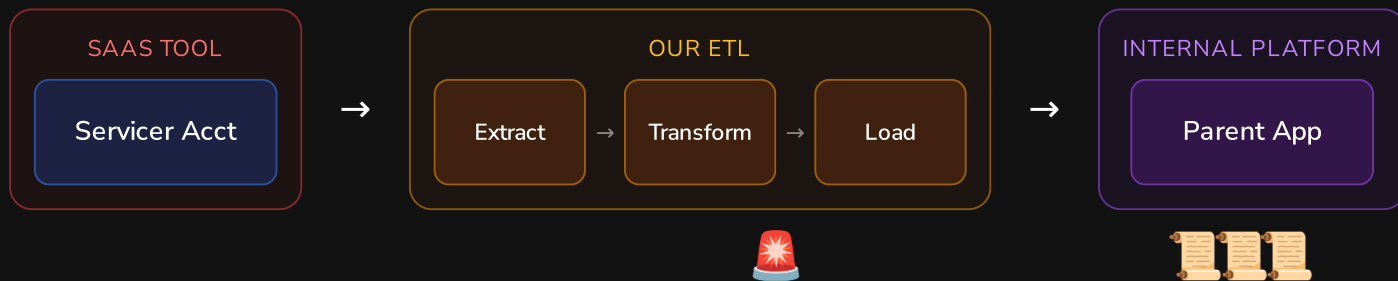
Data Quality Skepticism



Data Quality Skepticism



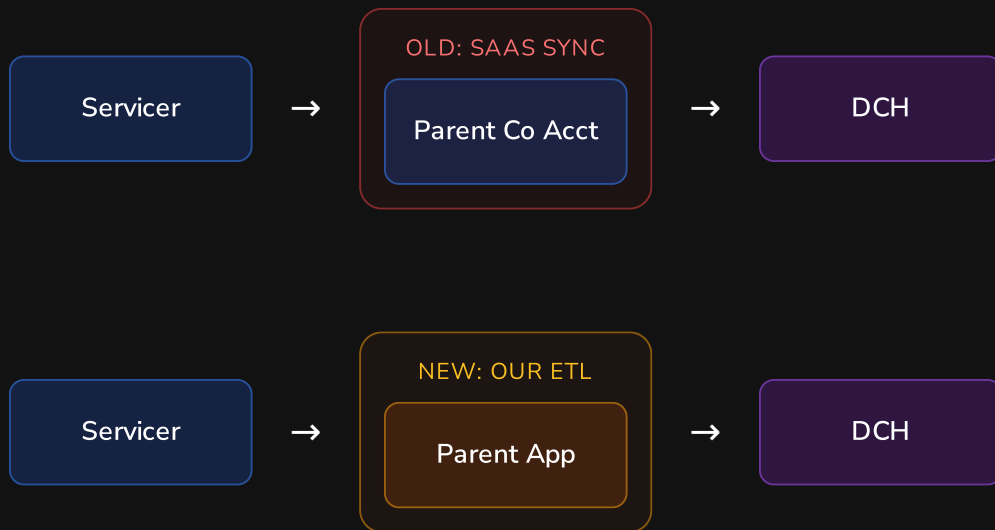
Data Quality Skepticism



Trust nothing from the third party. Validate at the boundary.

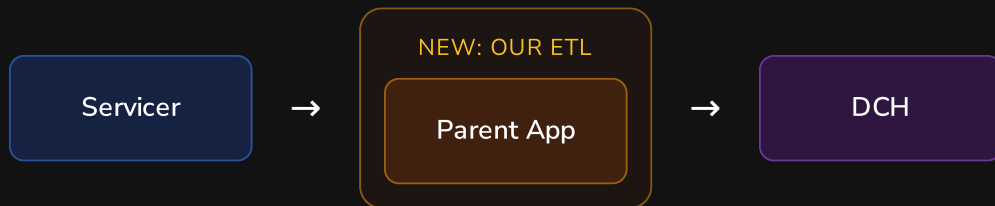
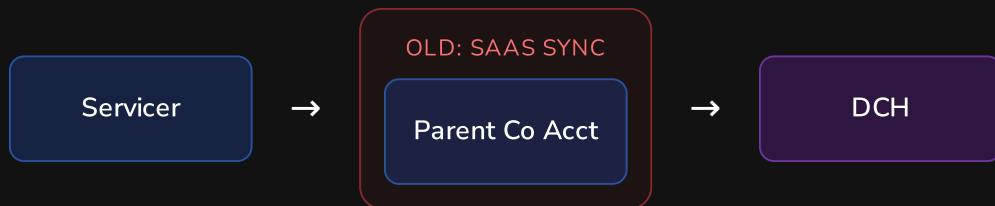


Swappable Replacement



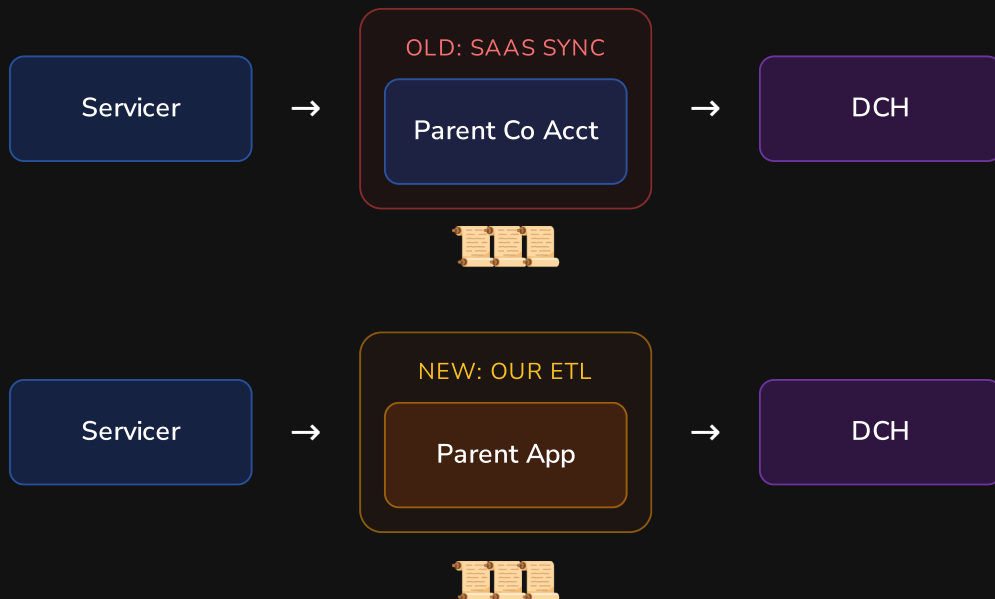


Swappable Replacement



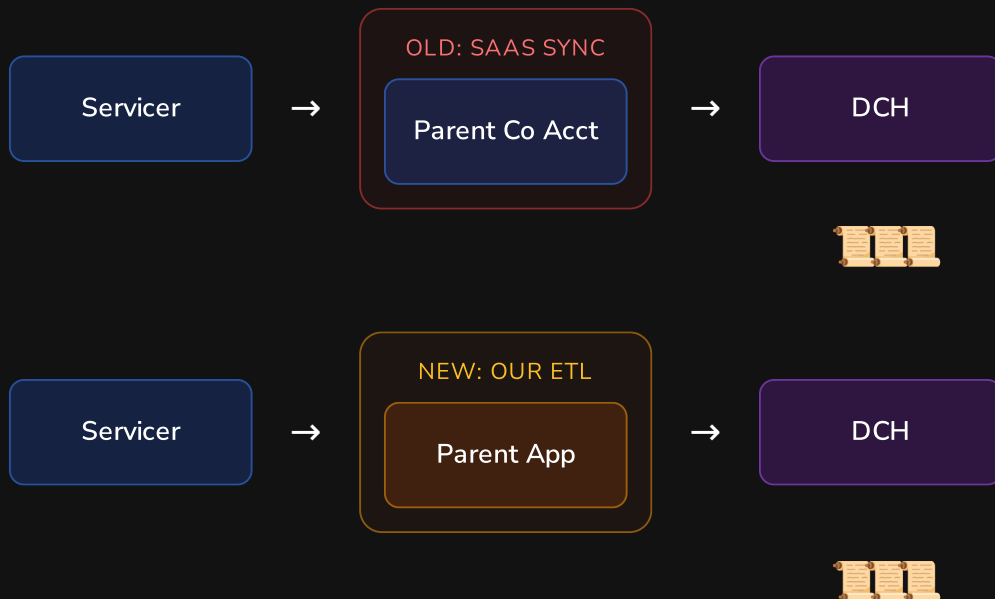


Swappable Replacement



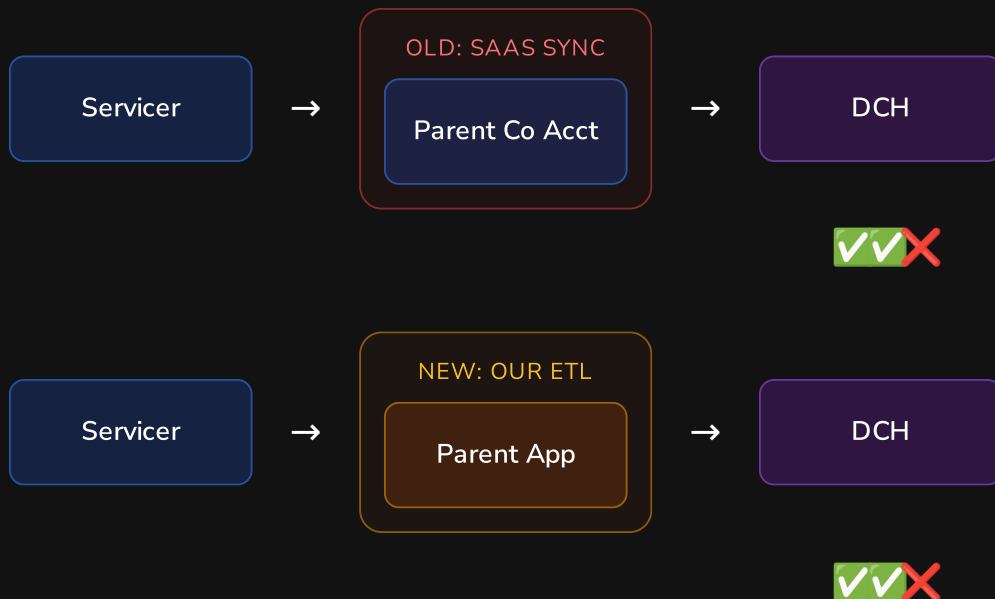


Swappable Replacement



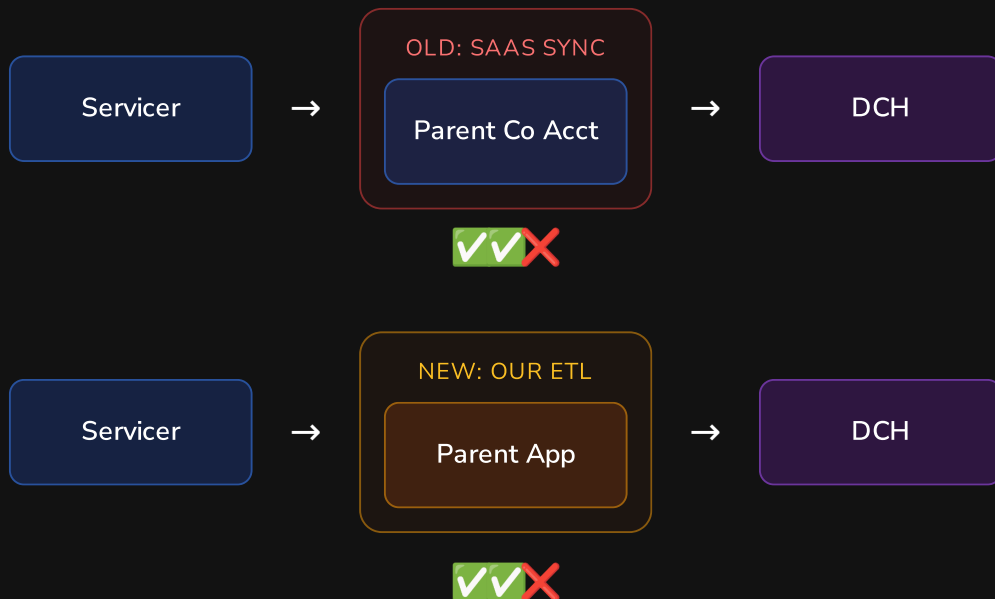


Swappable Replacement



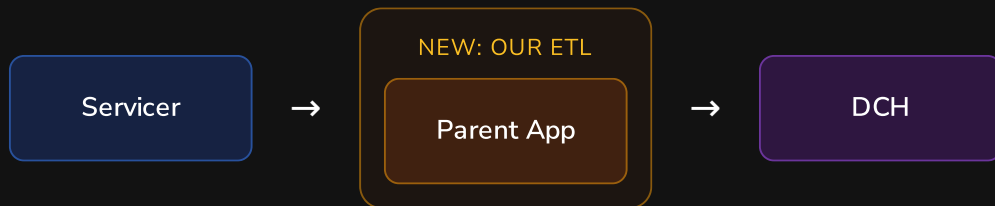
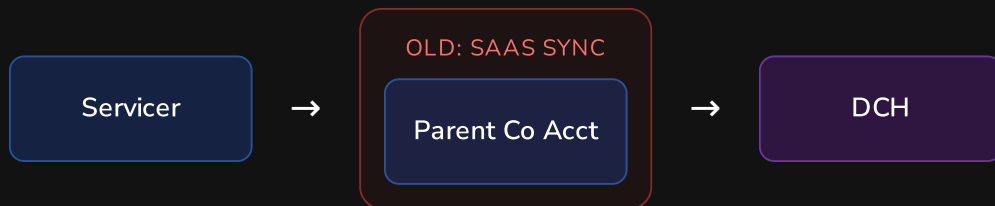


Swappable Replacement

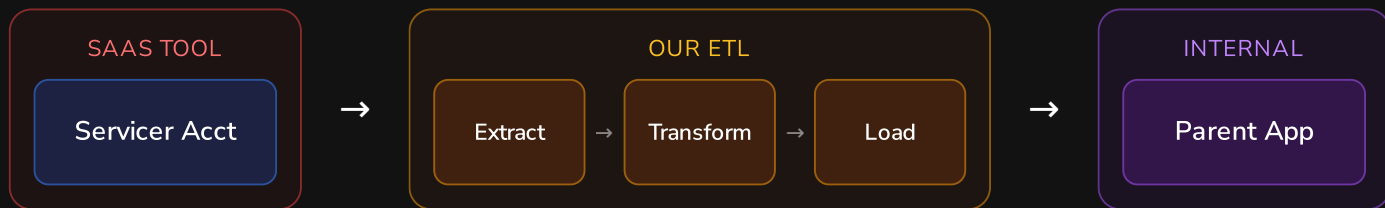




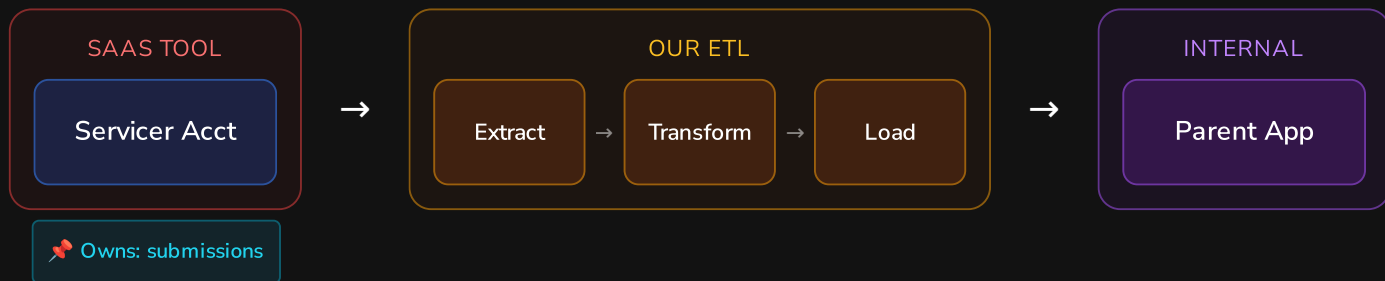
Swappable Replacement



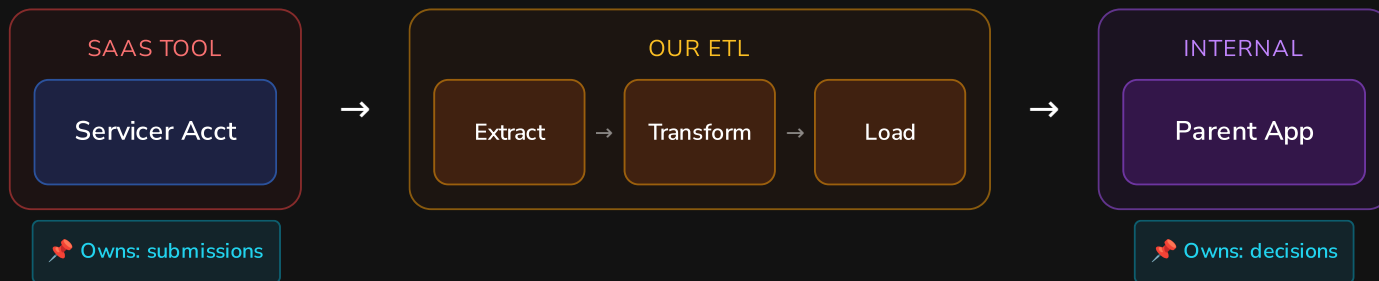
Clear Source of Truth



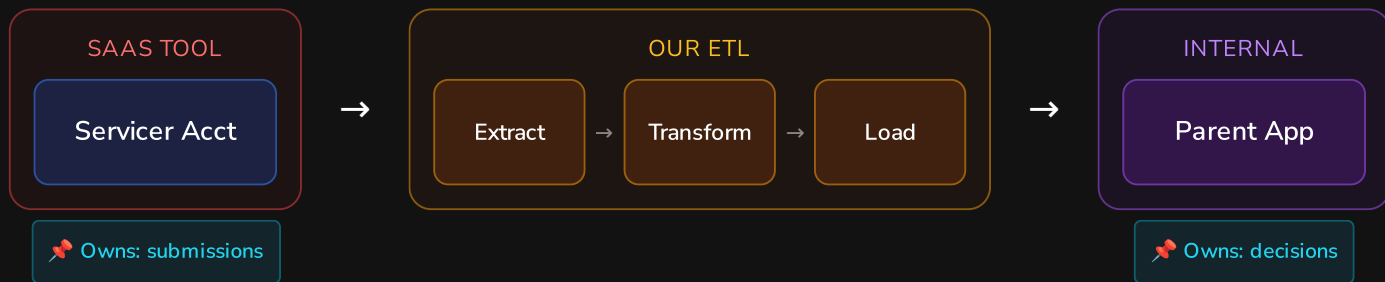
Clear Source of Truth



Clear Source of Truth



Clear Source of Truth



One system owns each fact. Everyone else is a reader.

Observability First



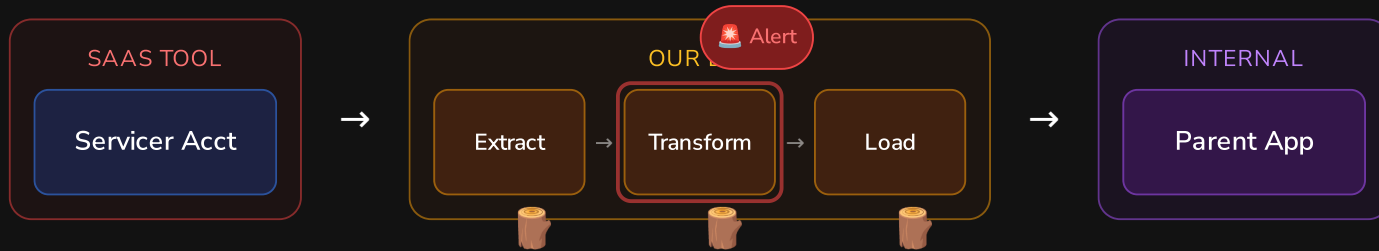
Observability First



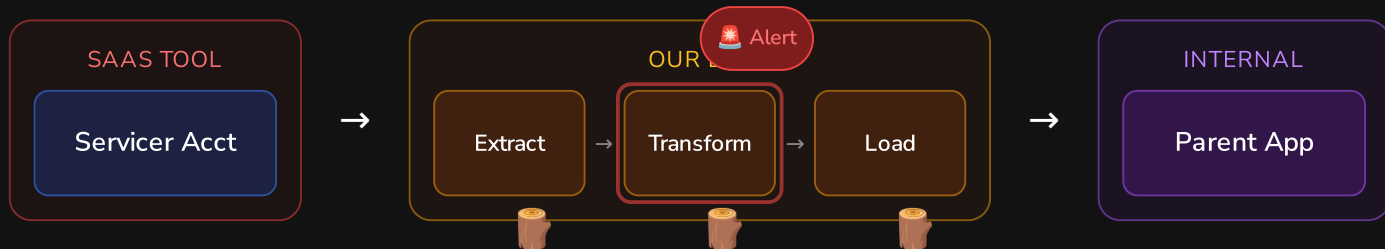
Observability First



Observability First



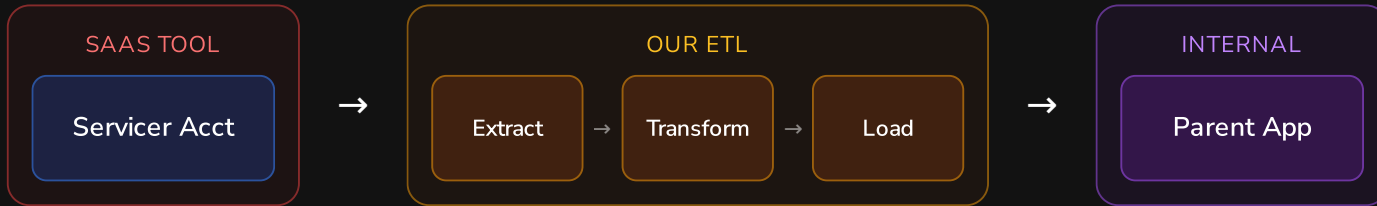
Observability First



Log everything. Alert on what matters. You can't fix what you can't see.



Idempotent Operations



Idempotent Operations





Idempotent Operations



Failures happen → need to retry

Idempotent Operations



 #1

 #2

Idempotent Operations



 #1

 #2

Idempotent Operations



✓ #1

✗ #2

The Launch

The Launch

 Shipped 2 months early

The Launch

 Shipped 2 months early

 Early production usage = early bug discovery

The Launch



Shipped 2 months early



Early production usage = early bug discovery



Users could test real workflows while we iterated

The Launch



Shipped 2 months early



Early production usage = early bug discovery



Users could test real workflows while we iterated

"And then... the phone rang."

Part 2: The Plot Twist

The 380K Document Problem

"Great news, we love the new system!"

"Great news, we love the new system!"

"Oh, by the way... we need our documents from the old system."

"Great news, we love the new system!"

"Oh, by the way... we need our documents from the old system."

 380,000 documents in the legacy system

"Great news, we love the new system!"

"Oh, by the way... we need our documents from the old system."

 380,000 documents in the legacy system

 No easy export

"Great news, we love the new system!"

"Oh, by the way... we need our documents from the old system."

 380,000 documents in the legacy system

 No easy export

 No API for bulk retrieval

"Great news, we love the new system!"

"Oh, by the way... we need our documents from the old system."

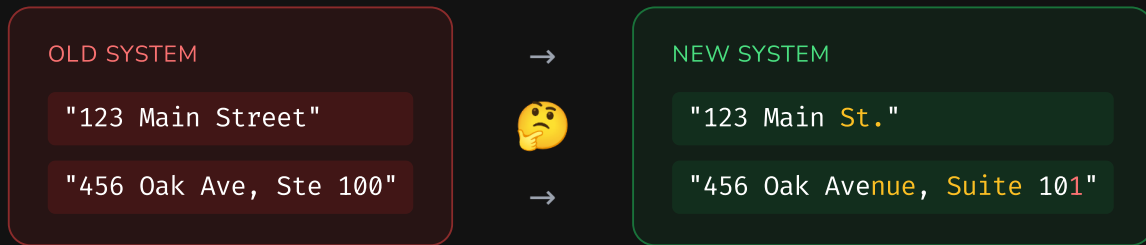
 380,000 documents in the legacy system

 No easy export

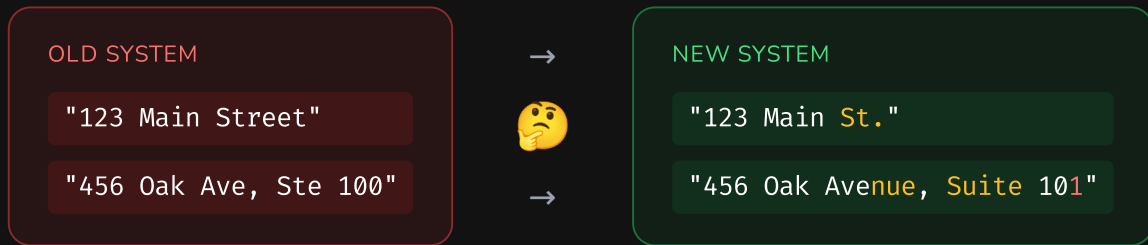
 No API for bulk retrieval

 The disincentivized vendor's help was required. Again.

The Mapping Problem

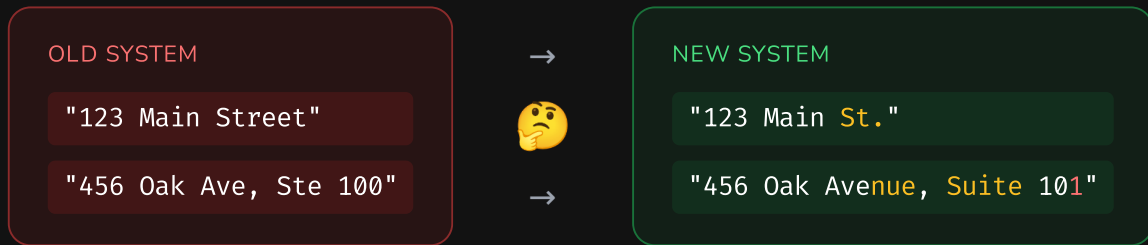


The Mapping Problem



No clean IDs. Fuzzy matching required.

The Mapping Problem



No clean IDs. Fuzzy matching required.

The save: ABDiff documentation let us pull in another engineer. The scope was already clear.

The Naive Approach

The Naive Approach

 Single-threaded upload

The Naive Approach

 Single-threaded upload

 Estimated time: 1 week

The Naive Approach

👉 Single-threaded upload

🕒 Estimated time: 1 week

😬 We didn't have 1 week.

The Performance Journey



Naive

~1 week

The Performance Journey



Naive

~1 week



The Performance Journey



Naive

~1 week



Multi-thread

Rate limited 

The Performance Journey



The Performance Journey



The Performance Journey



"Wait... the documents are **already in S3**. Why download and re-upload?"

"Ditching maintainability for a one-time script was liberating."

"Ditching maintainability for a one-time script was liberating."



Full retryability - resume from any point

"Ditching maintainability for a one-time script was liberating."



Full retryability - resume from any point



Automatic verification - validate what we ingested

"Ditching maintainability for a one-time script was liberating."



Full retryability - resume from any point



Automatic verification - validate what we ingested



Pierced the veil - checked S3 directly, bypassed API

"Ditching maintainability for a one-time script was liberating."



Full retryability - resume from any point



Automatic verification - validate what we ingested



Pierced the veil - checked S3 directly, bypassed API



Extensive logging for debugging

"Ditching maintainability for a one-time script was liberating."

-  Full retryability - resume from any point
-  Automatic verification - validate what we ingested
-  Pierced the veil - checked S3 directly, bypassed API
-  Extensive logging for debugging

"We safely cut tons of corners."

The Victory

The Victory



Documents ingested and verified 1 week early

The Victory



Documents ingested and verified 1 week early



Zero hitches

The Victory

 Documents ingested and verified 1 week early

 Zero hitches

 Contract cancelled on schedule